

5.20

1.LQVG 简介

LQVG 是一个专为遥感图像视觉定位（RSVG）任务设计的深度学习框架，以自然语言直接作为查询（Query），在复杂遥感图像中精准定位目标物体。模型需要在图像中完成定位，并输出边界框。

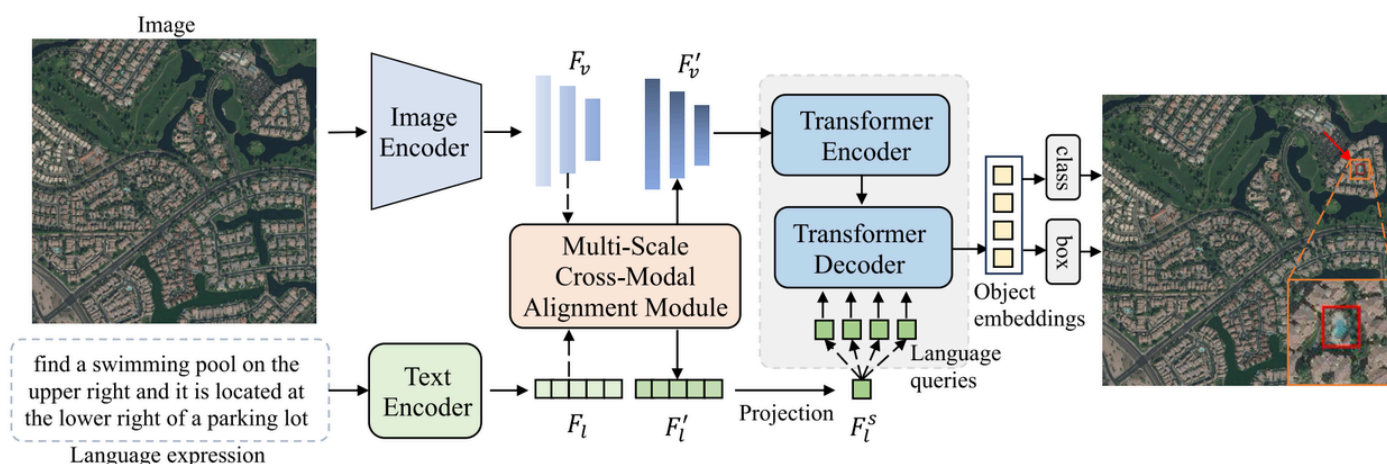


图1 LQVG 流程

以下是大致流程：

1. 图像与文本分别进行编码。
2. 将文本特征和视觉特征输入 MSCMA (Multiscale Cross-Modal Alignment 多尺度跨模态对齐) 部分。
3. 以语言特征作为 Q，单级视觉特征作为 K、V，进行交叉注意力，得到增强的文本特征。
4. 以单级视觉特征作为 Q，语言特征作为 K、V，进行交叉注意力，得到增强的多尺度视觉特征。
5. 将增强过后的文本特征池化为句级特征（这个句级特征指的是把原先一长串的文本特征浓缩成为一个单词，和[CLS] token 有点类似）。



第5步的形状变化：

增强过后的文本特征：[seq_len,B,256]

句级特征：[B,256]

6. 生成语言查询，这步将句级特征复制10份，为下一步在图像上预测10个框做准备。
7. 将增强的多尺度视觉特征、复制了10份的句级特征送入多模态 DETR 部分，

8. 预测头部分，根据上一步的输出得到10个查询的预测分数，以及这10个查询对应的框边界，选出得分最高的框计算损失，推理时作为结果。

2.具体做法

2.1 图像编码部分

图像编码采用了 Resnet-50，得到了多尺度的视觉特征。

代码块

```
1  ResNet 输出：
2  res2  [200×200]
3      |
4  res3  [100×100]  ← 第0层
5      |
6  res4  [50×50]    ← 第1层
7      |
8  res5  [25×25]    ← 第2层
9
10  res2/res3/res4/res5 是串行的，每个阶段都在前一个基础上继续下采样。
```

- res3 像素更多，细节更多，用于寻找小物体。
- res5 像素更少，更抽象，用于理解大物体和场景。

2.2 文本编码部分

采用模型为 RoBERTa-base，和 Resnet-50 一样，别人已经训练好了。

2.3 MSCMA 多尺度跨模态对齐

- 以语言特征作为 Q，单级视觉特征作为 K、V，进行交叉注意力，得到增强的文本特征。
- 以单级视觉特征作为 Q，语言特征作为 K、V，进行交叉注意力，得到增强的多尺度视觉特征。

$$A_{lv}^i = \text{Softmax} \left(\frac{F_l^i W^Q \cdot (F_v^i W^K)^T}{\sqrt{d_k}} \right)$$

$$F_l^{i+1} = (A_{lv}^i F_v^i W^V) \cdot F_l^i$$

图2.3.1 MSCMA 公式

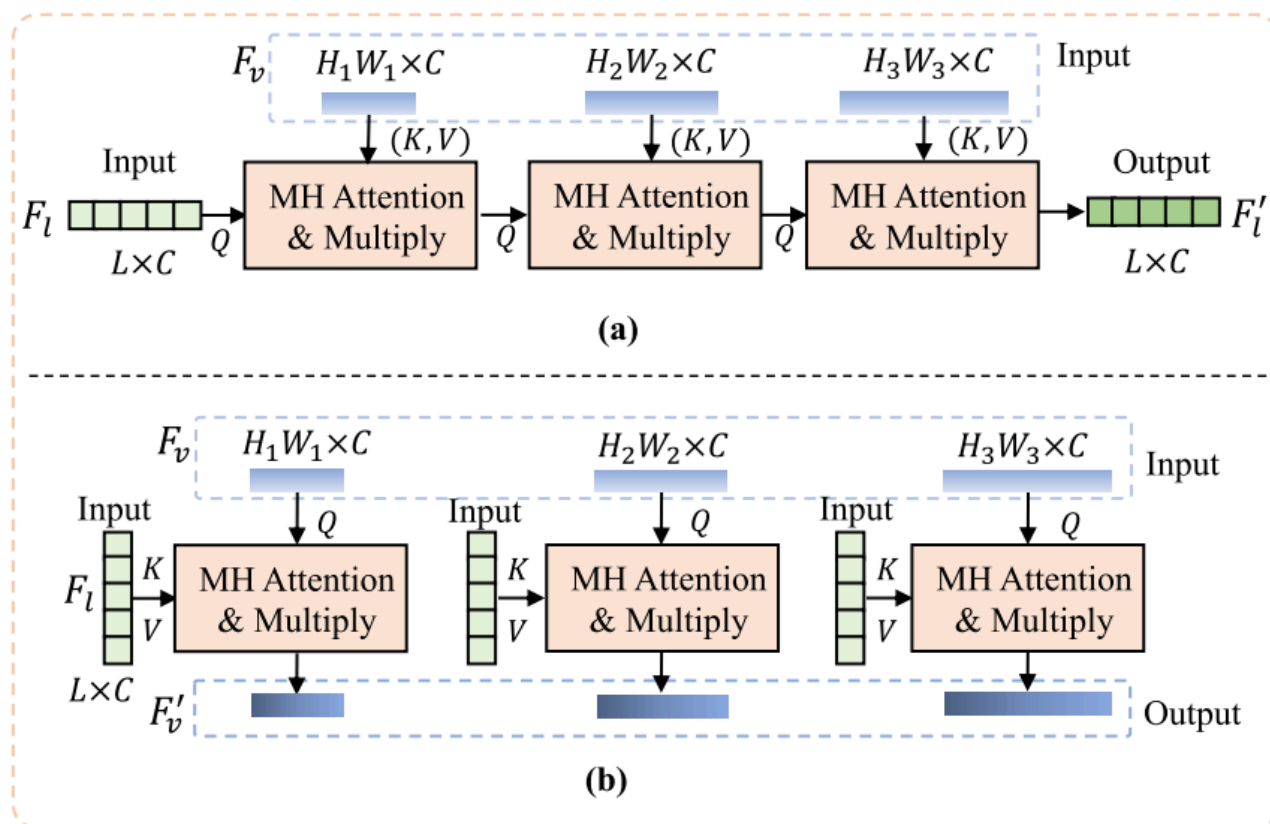


图2.3.2 (a) 语言-视觉交互 (文本特征作为查询) (b) 视觉-语言交互



为什么有三层？

视觉特征是多尺度的，有三张图，res3/res4/res5。

(a) 部分文本特征与三张图依次注意力交互。

(b) 部分视觉特征的三张图分别与文本特征进行交互，得到三个增强过后的视觉特征。

在注意力部分还有一个小细节：

```
def forward(self, tgt, memory,
            memory_key_padding_mask: Optional[Tensor] = None,
            pos: Optional[Tensor] = None,
            query_pos: Optional[Tensor] = None):
    tgt2 = self.multihead_attn(query=self.with_pos_embed(tgt, query_pos),
                              key=self.with_pos_embed(memory, pos),
                              value=memory, attn_mask=None,
                              key_padding_mask=memory_key_padding_mask)[0]
    tgt = tgt * tgt2
    # tgt = tgt + tgt2
    # tgt = self.norm(tgt + self.dropout(tgt2))
    return tgt
```

图2.3.3 视觉语言融合模块 官方代码



一般情况下注意力之后是要进行残差连接的，为什么这部分代码让输入与输出相乘？

首先，残差连接是为了训练深层次的网络，缓解梯度消失/爆炸的问题，这部分多头注意力的代码只会进行一层，所以残差连接也就没有必要。

其次，这行代码可以让注意力比较强的位置保留，注意力比较弱的位置信号减弱，起到去除噪声的效果。

2.4 句级特征提取

取出 [CLS] token，经过一层线性投影和 Tanh 激活函数。

2.5 多模态 DETR

提取出的句子特征复制十份，再加入一个可学习的位置嵌入，生成十个查询，这十个查询会送入多模态 DETR 的 Decoder 部分。



加入可学习的位置嵌入的目的是在图片的不同区域进行查询，同时对这十种查询有所区分。

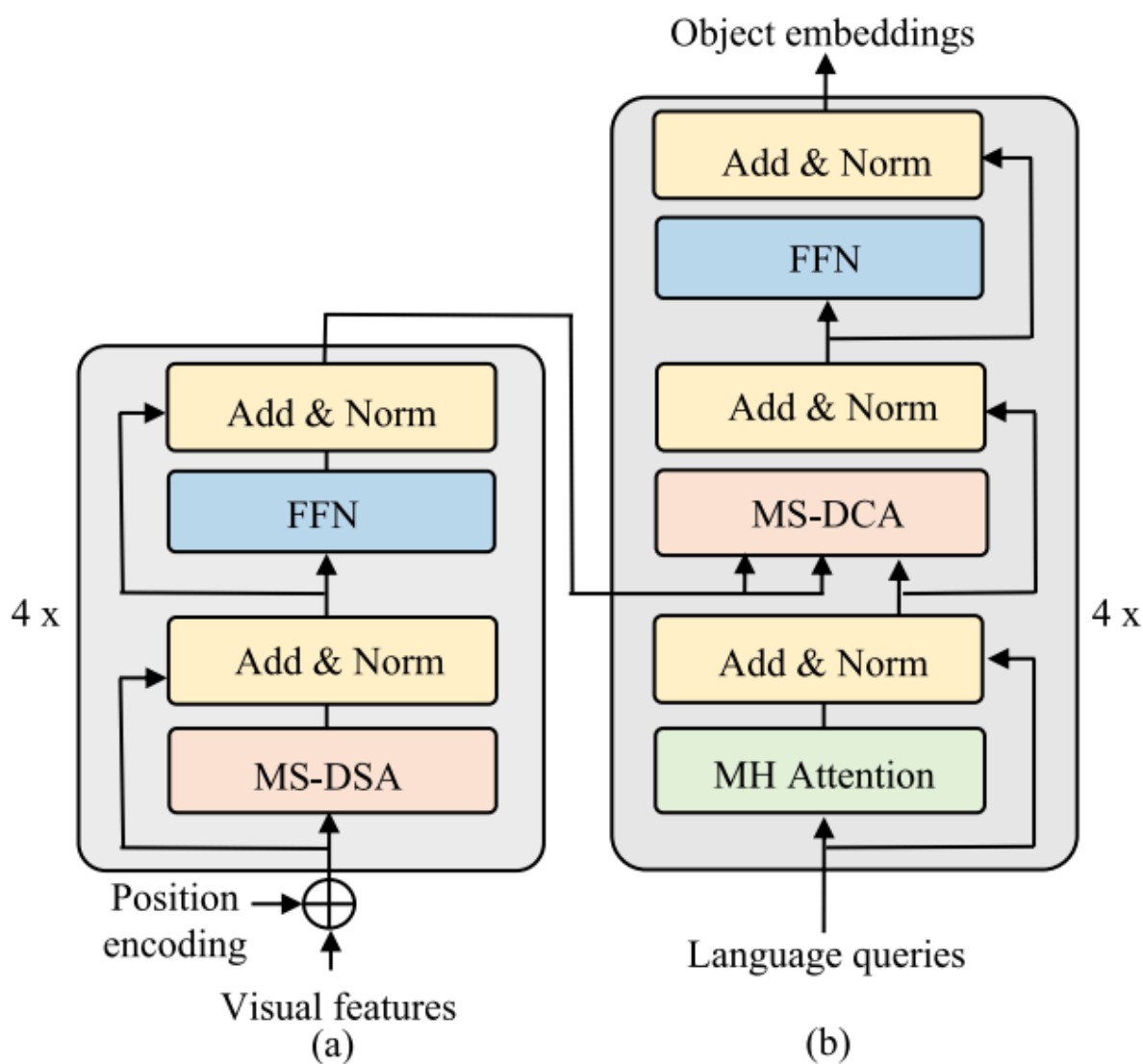


图2.5.1 多模态 DETR 的结构

- (a) 为 Encoder 部分
- (b) 为 Decoder 部分

接下来重点关注一下 MS-DCA 和 MS-DSA。

- MS-DSA (Multi-Scale Deformable Self-Attention) 多尺度可变形自注意力：

ViT 的编码器中，每一个 patch 都要对别的 patch 进行注意力乘积的运算，计算复杂度很高。

🏆 对于一张 $N * N$ 像素的图片，复杂度 $O(N^2)$ 。

而 MS-DSA 简化了这一过程，对于每一个像素，只在每一层上对4个采样点进行采样，这极大的加快了训练速度。

🎵 对于一张 $N * N$ 像素的图片，复杂度 $O(N)$ 。

具体做法。

首先为每一个像素生成一个参考点，对每个像素进行一次线性投影，把投影过后的特征维度拆分开，得到4个关于本参考点的偏移，本像素参考点的坐标分别加上预测过的偏移量，就得到了4个采样点的坐标，让本像素点与采样点分别做注意力运算即可，最后把所得结果拼起来。

形状变化

```
1 query [N, Len_q, 256] → Linear → [N, Len_q, 256]
2 [N, Len_q, 256] → reshape → [N, Len_q, 8头, 4层, 4点, 2]
3
4 # N 是 batch * 时间帧数, Len_q 是所有像素
5 # [N, Len_q, 8头, 4层, 4点, 2]
6 # 8头是多头注意力机制里面的8个头
7 # 4层是4层特征图片 (前面讲了3层 res3/res4/res5 实际代码实现上又下采样了一层 所以是4层)
8 # 4点是4个采样点
9 # 2代表坐标偏移量, Δx, Δy
```

在做注意力的时候，采用的公式和 Transformer 有所不同，简单讲一下：

$$\text{MS-DA}\left(z_q, \hat{p}_q, \{x^l\}_{l=1}^L\right) = \sum_{m=1}^M W_m \left[\sum_{l=1}^L \sum_{k=1}^K A_{mlqk} \cdot W'_m x^l(\phi_l(\hat{p}_q) + \Delta p_{mlqk}) \right] \quad (5)$$

图2.5.2 多尺度可变形注意力公式

1. $\phi_l(\hat{p}_q) + \Delta p_{mlqk}$ 为采样点位置。
 - ϕ_l 是一个坐标映射函数，把归一化的参考点 $\hat{p}_q$ 重新缩放到第 l 层特征图的真实坐标上。
2. $x^l(\phi_l(\hat{p}_q) + \Delta p_{mlqk})$ 为采样特征。
 - 采样点的坐标一般不是一个整数，取该位置附近的4个像素点来进行计算（每个像素对于采样特征的影响大小与距离有关），得到采样特征。
3. $A_{mlqk} \cdot W'_m x^l(\dots)$ 对采样特征进行变换与加权。
 - W'_m 是第 m 个头的输入投影矩阵，对采样到的特征向量进行线性变换。
 - A_{mlqk} 是注意力权重，一个正标量，由查询 z_q 和采样特征共同计算（Linear + softmax）。
4. $\sum_l \sum_k$ 合并不同特征图、不同采样点。
5. $\sum_m W_m[\dots]$ 合并注意力的每个头。

以上是 MS-DSA，它属于 Encoder 模块，所用到的数据全部属于视觉特征。



也不能说是全部属于视觉特征，因为在 MSCMA 部分，视觉特征与文本特征进行了交互。

- MS-DCA (Multi-Scale Deformable Cross-Attention) 多尺度可变形交叉注意力：

与 MS-DSA 结构类似，差别在于：

- 查询 query 是根据句级特征生成的十个语言查询。
- 编码器最终得到的输出作为 value 输入 MS-DCA。



公式中并没有 value，这个 value 实际上就是作为特征图。

感性认知一下，就是十个查询在编码器编码过后的图上开始打框。

2.6 预测头

历经千辛万苦，多模态 DETR 部分结束，输出了两个东西，十个分类分数，十个框。

简单二分类一下，判断哪些框是对的，最后输出。

2.7 损失计算

$$\mathcal{L}_{match}(y, \hat{y}_i) = \lambda_1 \mathcal{L}_{cls}(c, \hat{p}_i) + \lambda_2 \mathcal{L}_{L1}(b, \hat{b}_i) + \lambda_3 \mathcal{L}_G(b, \hat{b}_i)$$

图2.7.1 损失公式

L_{cls} 为分类损失。 L_{L1} 与 L_G 是预测框损失。

$$\lambda_1 = 2, \lambda_2 = 5, \lambda_3 = 2$$

图2.7.2 原论文配置的超参数

3.代码部分

仔细 debug 过官方代码，简单跑了一下。

配置

- 1 图片编码：ResNet-50
- 2 文本编码：RoBERTa-base

指标	训练前 (rsvg_mm权重)	训练后 (1 epoch)	提升
acc@0.5	34.77%	74.48%	+39.7
acc@0.6	33.21%	70.08%	+36.9
acc@0.7	31.12%	62.73%	+31.6
acc@0.8	27.23%	50.23%	+23.0
acc@0.9	18.59%	26.07%	+7.5
meanIoU	0.3186	0.6447	+0.326

图3.1 结果

训练前直接拿网上下载的权重跑了一遍推理，得到差劲的结果（表第2列）。

用 DIOR-RSVG 训练了一轮之后，得到的结果提升巨大，与原论文的指标相差没那么夸张了。（原论文指标比这个高百分之十几，可能再跑几轮就可以接近了）

受时间、设备限制，没能跑更多轮数。